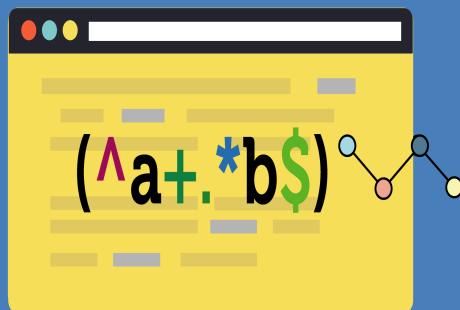


PYTHON

REGEX

A Little Guide



Python REGEX

A Little Guide

Scientific Programmer

This book is for sale at <http://leanpub.com/pythonregex>

This version was published on 2018-09-09



Leanpub

This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2018 Scientific Programmer

Contents

Python RegEx	1
Python regex match function	4
Python regex search function	8
Python regex match vs. search functions	12
Python regex group functions	14
Python regex sub function for search and replace	16
Python regex split function	18
Python regex findall function	19
Python regex compile function	21
Python regex finditer function	24
Python Regex - Lookarounds and Greedy Search	25
Python Lookahead	27
Python Look behind	28
Python Lazy and Greedy Search	29
Project 2: Parsing data from a HTML file with Python and REGEX	32
Project 3: PDF scraping in Python + REGEX	34
Project 4: Web scraping in Python + REGEX	37
Project 5: Amazon web crawling in Python + REGEX	40
Quiz # 1 - REGEX Patterns	42
Quiz # 2 Python REGEX Functions	45
References	47

Python RegEx

Hello coders! Let's start our quest with regular expressions (RegEx). In Python, the module `re` provides full support for Perl-like regular expressions in Python. We need to remember that there are many characters in Python, which would have special meaning when they are used in regular expression. To avoid bugs while dealing with regular expressions, we use raw strings as `r'expression'`.

The `re` module in Python provides multiple methods to perform queries on an input string. Here are the most commonly used methods:

- `re.match()`
- `re.search()`
- `re.split()`
- `re.sub()`
- `re.findall()`
- `re.compile()`

We will look at these function and related flags with examples in the next section.

Python Regular Expression Patterns List

The following table lists the regular expression syntax that is available in Python. Note that any Regex can be concatenated to form new regular expressions; if `X` and `Y` are both regular expressions, then `XY` is also a regular expression.

Pattern	Description
.	Matches any single character except newline. Using <code>m</code> option allows it to match newline as well.
^	Matches the start of the string, and in <code>re.MULTILINE</code> (see the next lesson on how to change to multiline) mode also matches immediately after each newline.
\$	Matches end of line. In <code>re.MULTILINE</code> mode also matches before a newline.
[.]	Matches any single character in brackets.
[^.]	Matches any single character not in brackets.
*	Matches 0 or more occurrences of preceding expression.
+	Matches 1 or more occurrence of preceding expression.
?	Matches 0 or 1 occurrence of preceding expression.
{n}	Matches exactly <i>n</i> number of occurrences of preceding expression.
{n,}	Matches <i>n</i> or more occurrences of preceding expression.
{n, m}	Matches at least <i>n</i> and at most <i>m</i> occurrences of preceding expression. For example, <code>x{3,5}</code> will match from 3 to 5 'x' characters.
Pattern	Description
x y	Matches either x or y.
\d	Matches digits. Equivalent to <code>[0-9]</code> .
\D	Matches nondigits.
\w	Matches word characters.
\W	Matches nonword characters.
\Z	Matches end of string.
\G	Matches point where last match finished.
\b	Matches the empty string, but only at the beginning or end of a word. Boundary between word and non-word and <code>/B</code> is opposite of <code>/b</code> . Example <code>r"\btwo\b"</code> for searching two from 'one two three'.

Pattern	Description
\B	Matches nonword boundaries.
\n, \t	Matches newlines, carriage returns, tabs, etc.
\s	Matches whitespace.
\S	Matches nonwhitespace.
\A	Matches beginning of string.
\Z	Matches end of string. If a newline exists, it matches just before newline.

Groups and Lookarounds

More details later:

Pattern	Description
(re)	Groups regular expressions and remembers matched text.
(?: re)	Groups regular expressions without remembering matched text. For example, the expression <code>(?:x{6})*</code> matches any multiple of six 'x' characters.
(?#...)	Comment.
(?= ...)	Matches if ... matches next, but doesn't consume any of the string. This is called a lookahead assertion . For example, <code>Scientific (?:Python)</code> will match <code>Scientific</code> only if it's followed by <code>Python</code> .
(?!...)	Matches if ... doesn't match next. This is a negative lookahead assertion .
(?<=...)	Matches if the current position in the string is preceded by a match for ... that ends at the current position.

Python regex `match` function

The `match` function attempts to match a `re` pattern to string with optional flags.

Here is the syntax for this function ââ

```
1 re.match(pattern, string, flags=0)
```

Where,

- `pattern` is the regular expression to be matched,
- `string` is the string to be searched to match the pattern at the beginning of string and
- `flags`, which you can specify different flags using bitwise OR (`|`).

Match Flags

Modifier	Description
<code>re.I</code>	Performs case-insensitive matching.
<code>re.L</code>	Interprets words according to the current locale. This interpretation affects the alphabetic group (<code>\w</code> and <code>\W</code>), as well as word boundary behavior (<code>\b</code> and <code>\B</code>).
<code>re.M</code>	Makes <code>\$</code> match the end of a line and makes <code>^</code> match the start of any line.
<code>re.S</code>	Makes a period (dot) match any character, including a newline.
<code>re.U</code>	Interprets letters according to the Unicode character set. This flag affects the behavior of <code>\w</code> , <code>\W</code> , <code>\b</code> , <code>\B</code> .
<code>re.X</code>	It ignores whitespace (except inside a set <code>[]</code> or when escaped by a backslash and treats unescaped <code>#</code> as a comment marker.

Return values

- The `re.match` function returns a `match` object on **success** and `None` upon failure. -
- Use `group(n)` or `groups()` function of `match` object to get matched expression, e.g., `group(n=0)` returns entire match (or specific subgroup `n`)
- The function `groups()` returns all matching subgroups in a tuple (empty if there weren't any).

Example 1

Let's find the words before and after the word `to`:

```
1  #!/usr/bin/python
2  import re
3
4  line = "Learn to Analyze Data with Scientific Python";
5
6  m = re.match( r'(.*) to (.*) .*', line, re.M|re.I)
7
8  if m:
9      print "m.group() : ", m.group()
10     print "m.group(1) : ", m.group(1)
11     print "m.group(2) : ", m.group(2)
12 else:
13     print "No match!!"
```

The first group `(.*)` identified the string: `Learn` and the next group `(.*)` identified the string: `Analyze`. Output:


```
1 m.group() : Learn to Analyze Data with Scientific Python
2 m.group(1) : Learn
3 m.group(2) : Analyze
```

Example 2

`groups([default])` returns a tuple containing all the subgroups of the match, from 1 up to however many groups are in the pattern.

```
1 #!/usr/bin/python
2 import re
3
4 line = "Learn Data, Python";
5
6 m = re.match( r'(\w+) (\w+)', line, re.M|re.I)
7
8 if m:
9     print "m.group() : ", m.groups()
10    print "m.group (1,2)", m.group(1, 2)
11 else:
12    print "No match!!"
```

Output:

```
1 m.group() : ('Learn', 'Data')
2 m.group (1,2) ('Learn', 'Data')
```

Example 3

`groupdict([default])` returns a dictionary containing all the named subgroups of the match, keyed by the subgroup name.

```
1  #!/usr/bin/python
2  import re
3
4  number = "124.13";
5
6  m = re.match( r'(?P<Expotent>\d+)\. (?P<Fraction>\d+)', nu\
7  mber)
8
9  if m:
10     print "m.groupdict() : ", m.groupdict()
11 else:
12     print "No match!!"
```

Output: m.groupdict() : {'Expotent': '124', 'Fraction': '13'}

Example 4

Start, end. How can we match the start or end of a string? We can use the “A” and “Z” metacharacters. We precede them with a backslash. We match strings that start with a certain letter, and those that end with another.

```
1  import re
2
3  values = ["Learn", "Live", "Python"];
4
5  for value in values:
6      # Match the start of a string.
7      result = re.match("\AL+", value)
8      if result:
9          print("START MATCH [L]:", value)
10
11     # Match the end of a string.
12     result2 = re.match(".+n\Z", value)
```

```
13     if result2:
14         print("END MATCH [n]:", value)
```

Output:

```
1  output
2
3  ('START MATCH [L]:', 'Learn')
4  ('END MATCH [n]:', 'Learn')
5  ('START MATCH [L]:', 'Live')
6  ('END MATCH [n]:', 'Python')
```

Example 5

`start([group])` and `end([group])` return the indices of the start and end of the substring matched by group. See the next lesson for an example.

Python regex search function

The Search Function

The search function searches for first occurrence of a re pattern to string with optional flags.

Here is the syntax for this function ^

```
1 re.search(pattern, string, flags=0)
```

Where, * `pattern` is the regular expression to be matched, * `string` is the string to be searched to match the pattern at the beginning of string and * `flags`, which you can specify different flags using bitwise OR (|).

Match Flags

Modifier	Description
<code>re.I</code>	Performs case-insensitive matching.
<code>re.L</code>	Interprets words according to the current locale. This interpretation affects the alphabetic group (<code>\w</code> and <code>\W</code>), as well as word boundary behavior (<code>\b</code> and <code>\B</code>).
<code>re.M</code>	Makes <code>\$</code> match the end of a line and makes <code>^</code> match the start of any line.
<code>re.S</code>	Makes a period (dot) match any character, including a newline.
<code>re.U</code>	Interprets letters according to the Unicode character set. This flag affects the behavior of <code>\w</code> , <code>\W</code> , <code>\b</code> , <code>\B</code> .
<code>re.X</code>	It ignores whitespace (except inside a set <code>[]</code> or when escaped by a backslash and treats unescaped <code>#</code> as a comment marker.

Return values

- The `re.search` function returns a match object on **success** and `None` upon failure. -
- Use `group(n)` or `groups()` function of match object to get matched expression, e.g., `group(n=0)` returns entire match (or specific subgroup `n`)
- The function `groups()` returns all matching subgroups in a tuple (empty if there weren't any).

Example 1

Let's find the words before and after the word to:

```
1  #!/usr/bin/python
2  import re
3
4  line = "Learn to Analyze Data with Scientific Python";
5
6  m = re.search( r'(.*) to (.*) .*', line, re.M|re.I)
7
8  if m:
9      print "m.group() : ", m.group()
10     print "m.group(1) : ", m.group(1)
11     print "m.group(2) : ", m.group(2)
12 else:
13     print "No match!!"
```

Output

```
1  m.group() :  Learn to Analyze Data with Scientific Python
2  m.group(1) :  Learn
3  m.group(2) :  Analyze
```

The first group (`.*`) identified the string: Learn and the next group (`.*?`) identified the string: Analyze.

Example 2

`groups([default])` returns a tuple containing all the subgroups of the match, from 1 up to however many groups are in the pattern.

```
1  #!/usr/bin/python
2  import re
3
4  line = "Learn Data, Python";
5
6  m = re.search( r'(\w+) (\w+)', line, re.M|re.I)
7
8  if m:
9      print "m.group() : ", m.groups()
10     print "m.group (1,2)", m.group(1, 2)
11 else:
12     print "No match!!"
```

Output m.group() : ('Learn', 'Data') m.group (1,2) ('Learn', 'Data')

Example 3

groupdict([default]) returns a dictionary containing all the named subgroups of the match, keyed by the subgroup name.

```
1  #!/usr/bin/python
2  import re
3
4  number = "124.13";
5
6  m = re.search( r'(?P<Exponent>\d+)\.(?P<Fraction>\d+)', n\
7  umber)
8
9  if m:
10     print "m.groupdict() : ", m.groupdict()
11 else:
12     print "No match!!"
```

```
output  m.groupdict() : {'Expotent': '124', 'Fraction':  
'13'}
```

Python regex `match` VS. `search` functions

We have learned so far that Python offers two different **primitive** operations:

- `match`
- `search`

So, how they are different to each other?

Note that `match` checks for a match only at the beginning of a string, while `search` checks for a match anywhere in the string!

Example 1

Let's try to find the word Python:

```
1  #!/usr/bin/python  
2  import re  
3  
4  line = "Learn to Analyze Data with Scientific Python";  
5  
6  m = re.search( r'(python)', line, re.M|re.I)  
7  
8  if m:  
9      print "m.group() : ", m.group()  
10 else:  
11     print "No match by obj.search!!"  
12
```

```
13 m = re.match( r'(python)', line, re.M|re.I )
14
15 if m:
16     print "m.group() : ", m.group()
17 else:
18     print "No match by obj.match"
```

Output

```
1 m.group() : Python
2 No match by obj.match
```

You see above that, match function won't find the word "Python", but search can! Also note the use of the re.I (case insensitive) option.

Example 2:

start([group]) and end([group]) return the indices of the start and end of the substring matched by group. We need to use search instead of match for this example:

```
1  #!/usr/bin/python
2  import re
3
4  email = "hello@leremove_thisarntoanalyzedata.com ";
5
6  # m = re.match ("remove_this", email) // This will not work!
7  rk!
8  m = re.search("remove_this", email)
9
10 if m:
11     print "email address : ", email[:m.start()] + email[m.\
12 end():]
13 else:
14     print "No match!!"
```


Output

```
1 email address : hello@learntoanalyzedata.com
```

Python regex group functions

A regular expression can have **named** groups. This makes it easier to retrieve those groups after calling `match()`. But it makes the pattern more complex.

Following example shows a named group (`first` and `last`).

```
1  #!/usr/bin/python
2  import re
3
4  # A string.
5  name = "Learn Scientific"
6
7  # Match with named groups.
8  m = re.match("(?P<first>\w+)\W+(?P<last>\w+)", name)
9
10 # Print groups using names as id.
11 if m:
12     print(m.group("first"))
13     print(m.group("last"))
```

Output:

```
1 Learn
2 Scientific
```

We can get the first name with the string `"first"` and the `group()` method. We use `"last"` for the last name.

Group dictionary `Groupdict`

A regular expression with named groups can fill a dictionary. This is done with the `groupdict()` method. In the dictionary, each group name is a key and Each value is the data matched by the regular expression. So we receive a key-value store based on groups.

```
1  import re
2
3  name = "Scientific Python"
4
5  # Match names.
6  m = re.match("(?P<first>\w+)\W+(?P<last>\w+)", name)
7
8  if m:
9      # Get dict.
10     d = m.groupdict()
11
12     # Loop over dictionary with for-loop.
13     for t in d:
14         print(" key:", t)
15         print("value:", d[t])
```

Output

```
1  (' key:', 'last')
2  ('value:', 'Python')
3  (' key:', 'first')
4  ('value:', 'Scientific')
```

Python regex `sub` function for search and replace

Python search and replace

The `sub()` function replaces every occurrence of a pattern with a string or the result of a function.

Syntax

```
1 re.sub(pattern, repl, string, maximum=0)
```

This method replaces all occurrences of the `re` pattern in `string` with `repl`, substituting all occurrences unless a `maximum` value is provided. Finally, returns the modified string.

Example 1

Format the phone number `+61-927 479-548`, remove everything except the digits:

```
1 #!/usr/bin/python
2 import re
3
4 phone = "Please call the phone # +61-927 479-548"
5
6 # Remove anything other than digits
7 num = re.sub(r'\D', "", phone)
8 print "The raw phone numbe is : ", num
```

Output

1 The raw phone numbe is : 61927479548

Example 2:

Let's use the `sub()` function to “munge” a text, i.e., **randomize** the order of all the characters in each word of a sentence except for the first and last characters:

```
1  #!/usr/bin/python
2  import re
3  import random
4
5  def repl(m):
6      inner_word = list(m.group(2))
7      random.shuffle(inner_word)
8      return m.group(1) + "".join(inner_word) + m.group\
9  (3)
10
11 line = "Learn Scientific Python with Regex";
12 m = re.sub(r"(\w)(\w+)(\w)", repl, line);
13
14 if m:
15     print "munged : ", m;
16 else:
17     print "No match!!";
```

Output

1 munged : Laren Scifneitic Phtoy **with** Rgeex

Python regex `split` function

Python string splitter

The `split()` function, splits a string by the occurrences of a pattern.

Syntax

```
1 re.split(pattern, string, maxsplit=0, flags=0)
```

If `maxsplit` is nonzero, at most `maxsplit` splits occur, and the remainder of the string is returned as the final element of the list.

Example 1

Break the string: 'Learn, Scientific, Python', into three elements:
Learn, Scientific, Python:

```
1 #!/usr/bin/python
2 import re
3
4 line = "Learn, Scientific, Python"
5
6 m = re.split('\W+', line)
7
8 if m:
9     print m
10 else:
11     print "No match!"
```

Output ['Learn', 'Scientific', 'Python']

Example 2

Now let's make a second example, where a splitter can any alphabet from [a-z]

```
1  #!/usr/bin/python
2  import re
3
4  line = "+61Lean7489Scientific324234"
5
6  m = re.split('[A-Za-z]+', line, re.I)
7
8  if m:
9      print m
10 else:
11     print "No match!"
```

Output ['+61', '7489', '324234']

Python regex findall function

Python string findall

`findall()` is a powerful function in the `re` module. It finds *all* the matches and returns them as a list of strings, with each string representing one match.

Syntax

```
1  re.findall(pattern, string, flags=0)
```

The string is scanned **left-to-right**, and matches are returned in the order found. If **one or more** groups are present in the pattern,

return a list of groups. Empty matches are included in the result unless they touch the beginning of another match.

Example 1

Find all and return the email addresses:

```
1  #!/usr/bin/python
2  import re
3
4  line = 'your alpha@scientificprograming.io, blah beta@sci\
5  entificprogramming.io blah user'
6
7  emails = re.findall(r'[\w\.-]+@[\w\.-]+', line)
8
9  if emails:
10     print emails
11 else:
12     print "No match!"
```

Output ['alpha@scientificprograming.io', 'beta@scientificprogramming.io']

Example 2: findall and Groups

Now let's make a second example. Groups () can be combined with findall(). If the pattern includes 2 or more parenthesis groups, then instead of returning a list of strings, findall() returns a list of *tuples*. Each tuple represents one match of the pattern, and inside the tuple is the group(1), group(2), etc.

The following example, will find, 'alpha', 'scientificprograming.io', 'beta', and 'scientificprogramming.me'.

```
1  #!/usr/bin/python
2  import re
3
4  line = 'your alpha@scientificprograming.io, blah beta@sci\
5  entificprogramming.me blah user'
6
7  tuples = re.findall(r'([\w\.-]+)([\w\.-]+)', line)
8
9  if tuples:
10     print tuples
11 else:
12     print "No match!"
```

Output [('alpha', 'scientificprograming.io'), ('beta', 'scientificprogramm

Python regex compile function

The compile function compiles a regular expression pattern into a regular expression object, which can be used for matching using its `match()`, `search()`, etc. methods.

Syntax

```
1  re.compile(pattern, flags=0)
```

The expression's behaviour can be modified by specifying a flags value (discussed earlier). Values can be any of the following variables, combined using bitwise OR (the `|` operator).

For example:


```
1 m = re.match(pattern, string)
```

is equivalent to:

```
1 p = re.compile(pattern)
2 m = p.match(string)
```

Note that the programs that use only a **few regular expressions at a time** don't need to compile regular expressions (recent patterns are cached automatically due to `re._MAXCACHE` setting).

Example

Consider that you have an html file `index.html` like below:

```
1 <html>
2 <header>SP: </header>
3 <body>
4 <h1>Learn</h1>
5 <p>Scientific Programming</p>
6 </body>
7 </html>
```

You want to read this file and output:

```
1 SP:Learn Scientific Programming
```

The following code can do this:

```
1  import re
2  import os
3  def main():
4      f = open('index.html')
5      pattern = re.compile(r'(?P<start><.+?>)(?P<content>.*\
6  ?)(</.+?>)' )
7      output_text = []
8      for text in f:
9          match = pattern.match(text)
10         if match is not None:
11             output_text.append(match.group('content'))
12
13     fixed_content = ' '.join(output_text)
14
15     print fixed_content
16
17     f.close()
18
19 if __name__ == '__main__':
20     main()
```

Output

1 SP: Learn Scientific Programming

Learning Tasks

- How the regex pattern `(?P<start><.+?>)(?P<content>.*\n?)(</.+?>)` matches the string `<h1>Learn</h1>`.
- Learn the use of the Python `.append` and `.join` functions.
Hints: `join()` returns a string in which the string elements of sequence have been joined by str separator and `append()` Add an item to the end of the list; equivalent to `a[len(a):] = [x]`.

Python regex `finditer` function

Python string `finditer`

`finditer()` is a powerful function in the `re` module. It returns an iterator yielding `MatchObject` instances over all non-overlapping matches for the RE pattern in string.

Syntax

```
1 re.finditer(pattern, string, flags=0)
```

Here the string is scanned **left-to-right**, and matches are returned in the order found. Empty matches are included in the result unless they touch the beginning of another match.

Example 1

Here is a simple example which demonstrates the use of `finditer`. It reads in a page of html text, finds all the occurrences of the word “**the**” and prints “the” and the following word. It also prints the character position of each match using the `MatchObject`’s `start()` method.

```
1 import re
2 import urllib2
3
4 html = urllib2.urlopen('https://docs.python.org/2/library\
5 /re.html').read()
6 pattern = r'\b(the\s+\w+)\s+'
7 regex = re.compile(pattern, re.IGNORECASE)
8 for match in regex.finditer(html):
9     print "%s: %s" % (match.start(), match.group(1))
```

Once you have the list of tuples, you can loop over it to do some computation for each tuple.

Expected output:

```
1 output
2
3 3261: The Python
4 4210: the backslash
5 4451: the same
6 4474: the same
7 4651: the pattern
8 4679: the regular
9 4930: The solution
10 5937: The functions
11 6301: the standard
12 and so on...
```

Python Regex - Lookarounds and Greedy Search

Lookarounds often cause confusion to new regex learners. There are four lookarounds:

- 1
- (?<= ...) and (?= ...),
- 2
- 3
- (?<! ...) and (?! ...)

Collectively, lookbehinds and lookaheads are known as lookarounds. Let’s see the following table of examples:

Lookaround	Name	What it Does
(?=learn)	Lookahead	Asserts that what immediately follows the current position in the string is learn
(?<=learn)	Lookbehind	Asserts that what immediately precedes the current position in the string is learn
(?!learn)	Negative Lookahead	Asserts that what immediately follows the current position in the string is not learn
(?<!learn)	Negative Lookbehind	Asserts that what immediately precedes the current position in the string is not learn

Python Lookahead

Python positive lookahead matches at a position where the pattern inside the lookahead can be matched. Matches only the position. It does not consume any characters or expand the match.

Example

Consider the following string:

```
begin:learner1:scientific:learner2:scientific:learner3:end
```

Positive lookahead assertion can help us to find all words followed by the word `scientific`.

```
1 import re
2
3 string = "begin:learner1:scientific:learner2:scientific:l\
4 earner3:end"
5 print re.findall(r"(\w+)(?=:scientific)", string)
```

Output

```
1 ['learner1', 'learner2']
```

Note the output `learner1` and `learner2`, but not `learner3`, which is followed by the word `:end`.

Negative Lookahead

Similar to positive lookahead, except that negative lookahead only succeeds if the regex inside the lookahead fails to match.

Example

Let's now proceed to an example, where we find the word (learner3) followed by end.

```
1 import re
2
3 string = "begin:learner1:scientific:learner2:scientific:l\
4 earner3:end"
5 print re.findall(r"(learner\d+)(?!:scientific)", string)
```

Output

```
1 ['learner3']
```

This matched all the words, not followed by the word scientific!

Python Look behind

Positive Lookbehind

(?<=regex) Matches at a position if the pattern inside the lookbehind can be matched ending at that position.

Example

Consider the following string:

```
begin:learner1:scientific:learner2:scientific:learner3:end
```

Positive lookbehind assertion can help us to find all words 'scientific', 'scientific' and 'end' preceded by the words learner{1-3}.

```
1 import re
2
3 string = "begin:learner1:scientific:learner2:scientific:l\
4 earner3:end"
5 print re.findall(r"(?<=learner\d:)(\b\w*\b)", string)
```

Output ['scientific', 'scientific', 'end']

Neagative Lookbehind

Similar to positive lookbehind, `(?<!regex)` matches at a position if the pattern inside the lookbehind cannot be matched ending at that position.

Example

Let's now proceed to an example, where we find the word (begin), not preceded by the words `learner{1-3}`.

```
1 import re
2
3 string = "begin:learner1:scientific:learner2:scientific:l\
4 earner3:end"
5 print re.findall(r"^(?<!learner\d:)(\b\w*\b)", string)
```

Python Lazy and Greedy Search

There are times when you want to match a pttter only optionally! The `?` character flags the group that precedes it as an optional part of the pattern. For example, enter the following into the interactive shell:


```
1 import re
2
3 Regex = re.compile(r'(scientific )?programming')
4 m1 = Regex.search('Learn programming')
5 m2 = Regex.search('Learn scientific programming')
6
7 print m1.group()
8 print m2.group()
```

The output will be:

```
1 programming
2 scientific programming
```

This means that the `(scientific )?` part of the regular expression means that the pattern `scientific` (notice the white space!) is an **optional** group. The regex will match text that has zero instances or one instance of `scientific` in it. This is why the regex matches both `'programming'` and `'scientific programming'`.

Note that the `'*'`, `'+'`, and `'?'` qualifiers are all **greedy**; they match as much text as possible. Sometimes this behavior isn't desired; if the RE pattern `<.*>` is matched against `'<H1>Learn Scientific Programming</H1>'`, it will match the entire string, and not just `'<H1>'`. Adding `'?'` after the qualifier makes it perform the match in non-greedy or minimal fashion; as few characters as possible will be matched. Using `.*?` in the previous expression will match only `'<H1>'`.

Project 1: Fun with DNA (REGEX Look around)!

DNA is a sequence of bases, A, C, G, or T. They are translated into proteins 3-bases where each sequence is called a **codon**. There is

a special start codon ATG, and three stop codons, TGA, TAG, and TAA.
Example:

```
1 cgcgcatgcatgctgacaaacgtagcgcgcgcgc
```

An opening reading frame or **ORF** consists of a **start codon**, followed by some more codons, and ending with a **stop codon**. The above example has overlapping ORFs.

- ATGcatgctga and
- ATGctgacTAA.

The following pattern only finds the first ORF (atgcatgctga'). Since it consumes the first ORF, it also consumes the beginning of the second ORF.

```
1 from re import *
2
3 dna = 'cgcgcatgcatgctgacaaacgtagcgcgcgcgc'
4 dna = dna.lower()
5 orfpattern = r'(?x) ( atg (?: (!tga|tag|taa) ... ) * (?: tga\
6 |tag|taa) )'
7 print findall(orfpattern,dna)
```

output ['atgcatgctga']

We want to find an ORF without consuming it, we can use a **positive lookahead** assertion ((?= (atg)). We put the whole ORF pattern inside the lookahead and find the two atgcatgctga and atgctgactaa.

```
1  from re import *
2
3  dna = 'cgcgcatgcatgcatgacTAacgTAGcgcgcgcg'
4  dna = dna.lower()
5  orfpat = r'(?x) (?= ( atg  (?: (!tga|tag|taa) ... )* (?\\
6  :tga|tag|taa) ))'
7  s = findall(orfpat,dna)
8  if s:
9      print ', '.join(s)
```

Output

```
1  atgcatgcatgga, atgcatgactaa
```

This project **adopts** and **simplifies** the Splitsville examples (DNA) from Rex Dwyer's ipython [notebook](https://github.com/rexdwyer/Splitsville)¹.

Project 2: Parsing data from a HTML file with Python and REGEX

In this project, we want to extract tabular information from a HTML file (see below). Our goal is to extract information available between <td> and </td> except the first numerical index (1..6).

Consider the data.html file below:

¹<https://github.com/rexdwyer/Splitsville>

```

1  <html>
2  <head>
3  <style>
4  table, th, td {
5      border: 1px solid black;
6      border-collapse: collapse;
7  }
8  th, td {
9      padding: 5px;
10 }
11 th {
12     text-align: left;
13 }
14 </style>
15 </head>
16 <body>
17 <table style="width:100%">
18 <tr align="center"><td>1</td> <td>England</td> <td>Englis\
19 h</td></tr>
20 <tr align="center"><td>2</td> <td>Japan</td> <td>Japanese\
21 </td></tr>
22 <tr align="center"><td>3</td> <td>China</td> <td>Chinese<
23 /td></tr>
24 <tr align="center"><td>4</td> <td>Middle-east</td> <td>Ar\
25 abic</td></tr>
26 <tr align="center"><td>5</td> <td>India</td> <td>Hindi</t\
27 d></tr>
28 <tr align="center"><td>6</td> <td>Thailand</td> <td>Thai<
29 /td></tr>
30 </table>
31 </body>
32 </html>

```

If we load the HTML file onto a browser it should look like below:

1	England	English
2	Japan	Japanese
3	China	Chinese
4	Middle-east	Arabic
5	India	Hindi
6	Thailand	Thai

Table: HTML tabular data scraping

Solution

In this code, we first extract HTML data (`data.html`) and then find and extract the values from the HTML code. ““ import re

with open('data.html', 'r') as myfile: data=myfile.read().replace('\n',
“”)

```
result=re.findall(r'<td>\w+</td>\s<td>(\w+)</td>\s<td>(\w+)</td>',data)
print(result) ““
```

Output :

```
1  [('England', 'English'),
2  ('Japan', 'Japanese'),
3  ('China', 'Chinese'),
4  ('India', 'Hindi'),
5  ('Thailand', 'Thai')]
```

Project 3: PDF scraping in Python + REGEX

In this project we will use a pdf file (see the screenshot below) from the diabetes.org website. Our goal is to list all the equipment models developed by the manufacturers names containing the word tandem (case insensitive).

Insulin Pumps

[illegible]

Find all the product models by the manufacturer called Tandem

Input file

You can download the input file from here: [data.pdf²](#).

Solution

A complete explanation of the Python code is **out of the scope** for this course (hint: learn the Python module `pdfquery`). It should be easy enough for you to understand how we capture the `product_name` from the pdf file using bounding box function `LTTextLineHorizontal.in_bbox("40, 48, 181, 633")` and then iterate over the products and search using regex and then only print the Tandem Manufacturers.

²<http://main.diabetes.org/dforg/pdfs/2015/2015-cg-insulin-pumps.pdf>

```
1  import re
2
3  import pdfquery
4  from lxml import etree
5
6
7  PDF_FILE = 'data.pdf'
8
9  pdf = pdfquery.PDFQuery(PDF_FILE)
10 pdf.load()
11
12 product_info = []
13 page_count = len(pdf._pages)
14 for pg in range(page_count):
15     data = pdf.extract([
16         ('with_parent', 'LTPage[pageid="{0}"]'.format(pg+1\
17     )),
18         ('with_formatter', None),
19         ('product_name', 'LTTextLineHorizontal:in_bbox("4\
20 0, 48, 181, 633")'),
21     ])
22
23     for ix, pn in enumerate(sorted([d for d in data['product_name'] if d.text.strip()], key=lambda x: x.get('y0'),\
24 reverse=True))):
25         if ix % 2 == 0:
26             product_info.append({'Manufacturer': pn.text.\
27 strip(), 'page': pg, 'y_start': float(pn.get('y1')), 'y_end\
28 nd': float(pn.get('y1'))-150})
29         if ix > 0:
30             product_info[-2]['y_end'] = float(pn.get(\
31 'y0'))+10.0
32     else:
33         product_info[-1]['Model'] = pn.text.strip()
```

```
36 pdf.file.close()
37
38
39 for p in product_info:
40     s = p['Manufacturer']
41     m = re.search(r"Tandem",s,re.I)
42     if m:
43         print('Manufacturer: {}[Model {}]\n'.format(p['Ma\
44 nufacturer'],p['Model']))
```

We have preloaded the data onto educative.io's server and you should be able to run the code straight ahead and get the output as follows:

```
1 Manufacturer: Tandem Diabetes Care[Model T:flex]
2 Manufacturer: Tandem Diabetes Care[Model T:slim]
```

From this result we can see that there are two models `T:flex` and `T:slim` supplied by the manufacturer called 'Tandem Diabetes Care'. The problem solution has been adopted and simplified from the reddit user [insainodwayno](https://www.reddit.com/user/insainodwayno)³.

Project 4: Web scraping in Python + REGEX

Web scraping or web data extraction is data scraping used for extracting data from websites. In this project, we will extract tabular data from the Boone County Sheriff's Dept [website](http://www.showmeboone.com/sheriff/JailResidents/JailResidents.asp)⁴ of criminal records (see the image below) and then find all the people whose name start with the word "A".

³<https://www.reddit.com/user/insainodwayno>

⁴<http://www.showmeboone.com/sheriff/JailResidents/JailResidents.asp>



Boone County Sheriff's Department
 Boone County Sheriff's Department
 2121 County Dr.
 Columbia, MO 65202

Office (573) 875-1111
 Fax (573) 874-8933

Home Menu Help

Current Detainees of Boone County Jail

Search By Name

Last Name:

First Name:

Run Date: 08/20/17

232 Total Items		Page Size: 50		<< < > >>		Page 1 of 5	
Last Name	First Name	Middle Name	Sex	Race	Age	City	State
ACTON	ANTHONY	SEAN	M	B	25	COLUMBIA	MO
ADAM	OMER	SIRAJ	M	B	29	COLUMBIA	MO
ALEXANDER	CHARLES	CODY	M	W	23	COLUMBIA	MO
ALLEN	WILLIAM	LAMAR	M	B	55	COLUMBIA	MO
AVALOS-AVALOS	JOSE		M	H	19	STANN	MO
BAIMA	TERESA	MAE	F	W	46	COLUMBIA	MO

Scraping tabular data from the Boone Country Sherrif's Dept website

Solution

The problem solution uses [BeautifulSoup](https://www.crummy.com/software/BeautifulSoup/)⁵. A detailed explanation of the code is out-of-scope for this course (hint: read the BS docs). In this code, we first extract HTML data and format/convert into BS's table using BS's `BeautifulSoup()` function, then find and extract the table from the HTML code.

```

1  import re
2  from pprint import pprint
3  import csv
4  import requests
5  from BeautifulSoup import BeautifulSoup
6
7
8  url = 'http://www.showmeboone.com/sheriff/JailResidents/JailResidents.asp'
9
10 response = requests.get(url)

```

⁵<https://www.crummy.com/software/BeautifulSoup/>

```

11 html = response.content
12
13 soup = BeautifulSoup(html)
14 table = soup.find('tbody', attrs={'class': 'stripe'})
15
16 list_of_rows = []
17
18 for row in table.findAll('tr')[0:]:
19     list_of_cells = []
20     for cell in row.findAll('td'):
21         text = cell.text.replace('&nbsp;', ' ')
22         list_of_cells.append(text)
23     list_of_rows.append(list_of_cells)
24
25 for line in list_of_rows:
26     row = '\t'.join(str(i) for i in line) # python 2
27     s=row[0:5] # Select only the Last names (1st column)
28     m = re.search(r"^A",s,re.I)
29     if m:
30         print row

```

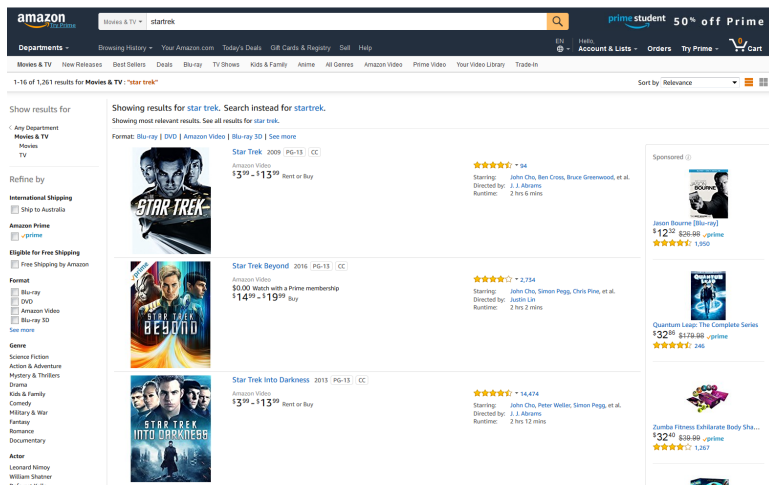
Expected output (Surnames started with the letter “A”): ACTON ANTHONY SEAN M B 25 COLUMBIA MO Details ADAM OMER SIRAJ M B 29 COLUMBIA MO Details ALEXANDER CHARLES CODY M W 23 COLUMBIA MO Details ALLEN WILLIAM LAMAR M B 55 COLUMBIA MO Details AVALOS-AVALOS JOSE M H 19 ST.ANN MO Details

This examples has been **adopted and extended** from the Python-BeautifulSoup’s [first web scraper](#)⁶, originally developed by Chase Davis, Jackie Kazil, Sisi Wei and Matt Wynn for bootcamps held by Investigative Reporters and Editors at the University of Missouri in Columbia, Missouri.

⁶<https://first-web-scraper.readthedocs.io/en/latest/>

Project 5: Amazon web crawling in Python + REGEX

A **Web crawler**, sometimes called a spider, is an Internet bot that systematically browses the World Wide Web, typically for the purpose of Web indexing (web spidering). In this project, we crawl the amazon.com⁷ website > Movies & TV > ‘startrek’ (see the image below). Then, we find the list of movies with ‘bonus’ content.



Amazon web scraping for Startrek DVD movies with 'bonus' content

Solution

The problem solution uses [BeautifulSoup](#)⁸. A detailed explanation of the code is out-of-scope for this course (hint: read the BS docs). In this code, we first extract HTML data and format/convert into BS's

⁷https://www.amazon.com/s/ref=nb_sb_noss_1?url=search-alias%3Dmovies-tv&field-keywords=startrek&rh=n%3A2625373011%2Ck%3Astartrek

⁸<https://www.crummy.com/software/BeautifulSoup/>

table using BS's BeautifulSoup() function, then find and extract the movies from the HTML code.

```

1  import re
2  from pprint import pprint
3  import csv
4  import requests
5
6
7  import requests
8  from bs4 import BeautifulSoup
9  def crawl_amazon_web(page,WebUrl):
10     if(page>0):
11         url = WebUrl
12         code = requests.get(url)
13         plain = code.text
14         s = BeautifulSoup(plain, "html.parser")
15
16         for link in s.findAll('a', {'class':'s-access-det\
17 ail-page'})):
18             movie_title = link.get('title')
19             m = re.search( r'Bonus',movie_title)
20             if m:
21                 print(movie_title)
22                 html_link = link.get('href')
23                 print(html_link)
24
25 crawl_amazon_web(1,'https://www.amazon.com/s/ref=nb_sb_no\
26 ss_2?url=search-alias%3Dmovies-tv&field-keywords=starwars\
27 &rh=n%3A2625373011%2Ck%3Astarwars')
```

Expected output (Startrek movies with bonus content): Star Wars:

The Force Awakens (Plus Bonus Features) <https://www.amazon.com/Star-Wars-For>
 Rogue One: A Star Wars Story (With Bonus Content) <https://www.amazon.com/Rogu>
 Easy!

This solution has been **adopted and extended** from the Dev.to [post](#)⁹ written by Pranay Das.

Quiz # 1 - REGEX Patterns

Challenge yourself with the Regex pattern quizzes.

Question 1 of 10

By default, a single dot (.) matches:

1. A single char
2. Nothing
3. Unlimited numbers of chars
4. A and B

Correct answer: 1

Question 2 of 10

Regex pattern `a+` matches

1. `b` and `aaab`
2. `ab` and `aaab`
3. `b` and `a+b`
4. `a+b`

Correct answer: 2

⁹<https://dev.to/pranay749254/build-a-simple-python-web-crawler>

Question 3 of 10

Regex `[0-9]+` matches one or more occurrence of any digit

1. True
2. False

Correct answer: 1

Question 4 of 10

`[^□□□` Matches any single character that is not in the class.

1. True
2. False

Correct answer: 1

Question 5 of 10

`^abc` matches - select multi answers

1. 123abc
2. abc123
3. aabc123
4. ^abc

Correct answer: 2 and 3

Question 6 of 10

The vertical bar separates two or more alternatives. A match occurs if any of the alternatives is satisfied. For example, `learn|scientific` matches

1. both learn and scientific
2. only learn

Correct answer: 1

Question 7 of 10

By default, regular expressions are case-sensitive.

1. True
2. False

Correct answer: 1

Question 8 of 10

When you put a plus sign (+) after something in a regular expression,

1. it indicates that the element may be repeated more than once.
2. it indicates summation

Correct answer: 1

Question 9 of 10

The star (*) has a similar meaning but also allows the pattern to match zero times.

1. True
2. False

Correct answer: 1

Question 10 of 10

Putting {4} after an element, \d{4}

1. Requires digits to occur exactly four times.
2. None
3. Four alphabets needs to be printed
4. Requires a mix of four alphabets and digits

Correct answer: 1

Quiz # 2 Python REGEX Functions

Challenge yourself with the Py Regex (re) module quizzes.

Question 1 of 5

When you deal with the HTML and XML you may need:

1. XML
2. Verbose
3. Non-greedy matching
4. HTML code

Correct answer: 3

Question 2 of 5

Split the string into a list, splitting it wherever the RE matches

1. `splitter()`
2. `sub()`
3. `splitn()`
4. `split()`

Correct answer: 4

Question 3 of 5

We use RE compile

1. All the time
2. Only when there are many regex(s)

Correct answer: 2

Question 4 of 5

What is (?= . . .) - Multiple correct

1. Positive lookahead assertion
2. Lookahead assertion
3. lookaround

4. Look-behind

Correct answers: 1-3

Question 4 of 5

Groups are marked by the metacharacters:

1. (and)
2. < and >
3. None
4. 1 and 2

Correct answers: 1

References

- **Python REGEX**
 - [Python Regular expression operations](#)¹⁰ - Official doco.
 - [Regular Expression HOWTO](#)¹¹ by A.M. Kuchling
- **Educative courses**
 - [Python 3: An interactive deep dive](#)¹² by Mark Pilgrim, implemented by the Educaitive Team
 - [Python 101: Interactively learn how to program with Python 3](#)¹³ by Michael Driscoll
 - [Python 201 - Interactively Learn Advanced Concepts in Python 3](#)¹⁴ by Michael Driscoll

¹⁰<https://docs.python.org/2/library/re.html>

¹¹<https://docs.python.org/2/howto/regex.html>

¹²<https://www.educative.io/collection/10370001/5705097937944576?authorName=Educative>

Educative

¹³<https://www.educative.io/collection/5663684521099264/5707702298738688?authorName=Michael%20Driscoll>

¹⁴<https://www.educative.io/collection/5663684521099264/5693417237512192?authorName=Michael%20Driscoll>

- **Online Python Courses**

- edx’s [Introduction to Computer Science and Programming Using Python](https://www.edx.org/course/introduction-computer-science-mitx-6-00-1x-0#.VJw5pv-kAA)¹⁵ . The companion book can be [found here](http://mitpress.mit.edu/books/introduction-computation-and-programming-using-python-0)¹⁶.
- MIT Open Courseware also offers a gentler “lead-in” course designed for those with no programming background that you can take before taking the above: [Building Programming Experience: A Lead-In to 6.001](http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-090-building-programming-experience-a-lead-in-to-6-001-january-iap-2005/)¹⁷.
- MIT Open Courseware’s [A Gentle Introduction to Programming using Python](http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-189-a-gentle-introduction-to-programming-using-python-january-iap-2011/)¹⁸
- Coursera’s [Programming for Everybody \(Python\)](https://www.coursera.org/course/pythonlearn)¹⁹ For beginners.
- Codecademy’s [beginners](http://www.codecademy.com/tracks/python); tends to focus primarily on syntax [Python track](https://www.codecademy.com/tracks/python)²⁰.
- Udacity’s [Programming Foundations with Python](https://www.udacity.com/course/ud036)²¹. Focuses on object-oriented programming.
- Team Treehouse’s [Python course](http://teamtreehouse.com/features/python)²².
- Udemy’s [Python Regular Expressions with Data Scraping Projects](https://www.udemy.com/python-regular-expressions-with-data-scraping-projects/)²³, this book - video version!

- **Interactive resources:**

- [LearnPython](http://learnpython.org)²⁴ An interactive online guide that teaches basic Python.
- [Try Python](http://www.trypython.org/)²⁵ Another interactive online guide.

¹⁵<https://www.edx.org/course/introduction-computer-science-mitx-6-00-1x-0#.VJw5pv-kAA>

¹⁶<http://mitpress.mit.edu/books/introduction-computation-and-programming-using-python-0>

¹⁷<http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-090-building-programming-experience-a-lead-in-to-6-001-january-iap-2005/>

¹⁸<http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-189-a-gentle-introduction-to-programming-using-python-january-iap-2011/>

¹⁹<https://www.coursera.org/course/pythonlearn>

²⁰<http://www.codecademy.com/tracks/python>

²¹<https://www.udacity.com/course/ud036>

²²<http://teamtreehouse.com/features/python>

²³<https://www.udemy.com/python-regular-expressions-with-data-scraping-projects/>

²⁴<http://learnpython.org>

²⁵<http://www.trypython.org/>

- Educative’s [Python Regular Expressions with Data Scraping Projects](#)²⁶, this book - interactive version!
- **Books and tutorials (online):**
 - [Learn Python the Hard Way](#)²⁷ Part of the “Learn X the Hard Way” series. Despite its name, this is one of the easiest introductions to Python available.
 - [Automate the Boring Stuff with Python](#)²⁸ .From the [Invent with Python](#)²⁹ author.
 - How to Think Like a Computer Scientist ([Python 2 version](#)³⁰ and [Python 3 version](#)³¹)
 - [Think Python](#)³² Comprehensive introductory text on Python.
 - The official Python tutorial (for [Python 2](#)³³ and [Python 3](#)³⁴). Moves a little quickly, but is very comprehensive and thorough.
 - [Problem Solving with Algorithms and Data Structures](#)³⁵
 - [Dive into Python 3](#)³⁶ An accelerated introduction to Python.
 - [Program Arcade Games With Python And Pygame](#)³⁷
 - [The Hitchhiker’s Guide to Python](#)³⁸
 - [pycrumbs](#)³⁹ A huge list of many useful articles, tutorials, and snippets on Python, ranging from basic to advanced.

²⁶<https://www.educative.io/collection/5183519089229824/5682462386552832>

²⁷<http://learnpythonthehardway.org/book/>

²⁸<http://automatetheboringstuff.com/>

²⁹<http://inventwithpython.com/>

³⁰<http://www.openbookproject.net/thinkcs/python/english2e/>

³¹<http://www.openbookproject.net/thinkcs/python/english3e/>

³²<http://www.greenteapress.com/thinkpython/>

³³<https://docs.python.org/2/tutorial/>

³⁴<https://docs.python.org/3/tutorial/>

³⁵<http://interactivepython.org/runestone/static/pythonds/index.html>

³⁶<http://www.diveintopython3.net/>

³⁷<http://ProgramArcadeGames.com>

³⁸<https://python-guide.readthedocs.org/en/latest/>

³⁹<http://resrc.io/list/4/pycrumbs/>

- [PyMOTW](#)⁴⁰ A tour of the Python standard library through short examples.
- [Import Python](#)⁴¹ A catalog of Python books (some are free)
- **Exercises:**
 - [Pyschools](#)⁴² Exercises and challenges in Python.

⁴⁰<http://pymotw.com/>

⁴¹<http://importpython.com/books/>

⁴²<http://www.pyschools.com/>